

**"AÑO DEL BICENTENARIO, DE LA CONSOLIDACIÓN DE
NUESTRA INDEPENDENCIA, Y DE LA CONMEMORACIÓN DE LAS
HEROICAS BATALLAS DE JUNÍN Y AYACUCHO"**

UNIVERSIDAD PERUANA LOS ANDES



UPLA

**ESCUELA PROFESIONAL DE INGENIERIA DE
SISTEMAS**

ASIGNATURA:

BASE DE DATOS 2

TEMA:

"RESOLUCION DE EJERCICIOS SEMANA 13"

DOCENTE : RAUL ENRIQUE FERNANDEZ BEJARANO

ESTUDIANTE : CUBA VILLANUEVA JAVIER AMILCAR

CICLO : 5to

Huancayo – 2024

Proyecto 1:

1. Enunciado del Ejercicio

La Oficina de Tecnologías de Información de la universidad ha detectado que el módulo de registro de expedientes de bachiller y título profesional presenta problemas de rendimiento durante las horas de mayor demanda.

Se requiere implementar una solución utilizando **Extended Events** de SQL Server para capturar automáticamente todas las consultas cuya duración sea superior a **1000 milisegundos** sobre la base de datos **GradosTitulos**.

```
1  --1.1
2  USE GradosTitulos;
3  GO
4
5  IF OBJECT_ID('dbo.HistoricoConsultasLentas') IS NOT NULL
6  DROP TABLE dbo.HistoricoConsultasLentas;
7  GO
8
9  CREATE TABLE dbo.HistoricoConsultasLentas
10 (
11     IdHistorial INT IDENTITY(1,1) PRIMARY KEY,
12     FechaCaptura DATETIME,
13     BaseDatos VARCHAR(100),
14     UsuarioSQL VARCHAR(100),
15     Aplicacion VARCHAR(200),
16     HostName VARCHAR(100),
17     Duracion_ms BIGINT,
18     CPU_ms BIGINT,
19     Lecturas BIGINT,
20     Escrituras BIGINT,
21     ConsultaSQL NVARCHAR(MAX)
22 );
23 GO
24
25 ALTER EVENT SESSION XE_ConsultasLentas
26 ON SERVER
27 STATE = START;
28 GO
29
30 SELECT
31     name,
32     startup_state
33 FROM sys.server_event_sessions
34 WHERE name='XE_ConsultasLentas';
35 GO
36
37 SELECT *
38 FROM sys.fn_xe_file_target_read_file
```

3. Justificación Técnica

Para este proyecto se utilizó **Extended Events**, ya que es la tecnología recomendada por Microsoft para el monitoreo de rendimiento en SQL Server, reemplazando en gran medida a SQL Profiler por su menor impacto sobre el servidor.

La solución implementada crea una sesión denominada XE_ConsultasLentas, la cual

registra únicamente las consultas cuya duración supera los **1000 milisegundos** dentro de la base de datos **GradosTitulos**. De esta forma se evita capturar eventos innecesarios y se reduce el consumo de recursos.

4. Explicación de las buenas prácticas utilizadas

Durante el desarrollo de la solución se aplicaron las siguientes buenas prácticas:

- **Uso de Extended Events:** Se empleó esta tecnología por ser más eficiente y generar menor impacto en el rendimiento del servidor que SQL Profiler.
- **Filtrado de eventos:** La sesión captura únicamente consultas con duración superior a 1000 ms y pertenecientes a la base de datos GradosTitulos, reduciendo el volumen de información recolectada.
- **Persistencia de la información:** Los eventos se almacenan en archivos .xel, lo que permite conservar evidencia para auditorías y análisis posteriores.
- **Historial centralizado:** Se creó una tabla de auditoría para almacenar los eventos procesados, facilitando la elaboración de reportes y el análisis histórico.

Proyecto 2:

1. Enunciado del ejercicio

La base de datos **GradosTitulos** presenta problemas de rendimiento ocasionados por bloqueos, deadlocks, consultas con alto consumo de CPU y errores durante la ejecución de los procesos académicos.

Se requiere implementar una sesión de **Extended Events** que permita registrar estos eventos críticos y almacenarlos en un archivo .xel para su posterior análisis.

```
98
99 --1.2
10
11 USE master;
12 GO
13
14 IF EXISTS(
15 SELECT *
16 FROM sys.server_event_sessions
17 WHERE name='XE_MonitoreoGeneral'
18 )
19 DROP EVENT SESSION XE_MonitoreoGeneral ON SERVER;
20 GO
21
22 CREATE EVENT SESSION XE_MonitoreoGeneral
23 ON SERVER
24
25 -- Deadlocks
26 ADD EVENT sqlserver.xml_deadlock_report
27
28 -- Bloqueos prolongados
29 ADD EVENT sqlserver.blocked_process_report
30
31 -- Errores de nivel 16 o superior
32 ADD EVENT sqlserver.error_reported
33 (
34     ACTION
35     (
36         sqlserver.database_name,
37         sqlserver.sql_text,
38         sqlserver.client_hostname,
39         sqlserver.username
40     )
41     WHERE severity>=16
42 )
43
44 -- Consultas finalizadas con alto consumo de CPU
45 ADD EVENT sqlserver.rpc_completed
46 (
47     ACTION
48     (
49         sqlserver.database_name,
50         sqlserver.sql_text,
51         sqlserver.client_hostname,
52         sqlserver.username
53     )
54 )
```

3. Justificación Técnica

Se utilizó **Extended Events** por ser la herramienta recomendada para el monitoreo de SQL Server, ya que consume menos recursos que SQL Profiler. La sesión creada permite registrar eventos importantes como deadlocks, bloqueos, errores de severidad alta y consultas con elevado consumo de CPU sobre la base de datos **GradosTitulos**. Los eventos se almacenan en archivos .xel, lo que facilita su consulta y análisis posterior mediante funciones nativas de SQL Server, permitiendo identificar rápidamente problemas de rendimiento y tomar acciones correctivas.

4. Explicación de las buenas prácticas utilizadas

- Se utilizó **Extended Events**, recomendado por Microsoft para el monitoreo de SQL Server.
- Se filtraron únicamente los eventos relevantes para reducir el consumo de recursos.
- Los eventos se almacenan en archivos .xel, facilitando auditorías posteriores.
- Se limitaron el tamaño y número de archivos mediante `max_file_size` y `max_rollover_files`.

Proyecto 3:

1. Enunciado del ejercicio

La Oficina de Tecnologías de Información requiere implementar un sistema permanente de monitoreo para la base de datos **GradosTitulos**, con el fin de registrar automáticamente las consultas lentas y generar reportes históricos que permitan analizar el rendimiento del sistema.

Se solicita crear una sesión de **Extended Events**, una tabla para almacenar el historial, un procedimiento almacenado para importar los eventos y un reporte de las consultas más costosas.

```
73 USE GradosTitulos;
74 GO
75
76 ✓ IF OBJECT_ID('dbo.HistorialMonitoreo') IS NOT NULL
77     DROP TABLE dbo.HistorialMonitoreo;
78 GO
79
80 ✓ CREATE TABLE dbo.HistorialMonitoreo
81 (
82     IdHistorial INT IDENTITY(1,1) PRIMARY KEY,
83     FechaEvento DATETIME,
84     BaseDatos VARCHAR(100),
85     UsuarioSQL VARCHAR(100),
86     HostName VARCHAR(100),
87     Duracion_ms BIGINT,
88     CPU_ms BIGINT,
89     ConsultaSQL NVARCHAR(MAX)
90 );
91 GO
92
93 |
94
95 ✓ --2.1
96
97 --2.2
```

% 13 0 ↑ ↓

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-07-13T20:00:40.2903326-05:00

3. Justificación Técnica

Se implementó una solución basada en **Extended Events** para realizar un monitoreo continuo del rendimiento de la base de datos **GradosTitulos**. La sesión registra automáticamente las consultas cuya duración supera un segundo y almacena los eventos en archivos .xel. Posteriormente, un procedimiento almacenado importa la información a una tabla histórica, permitiendo consultar el comportamiento del sistema a lo largo del tiempo y detectar consultas con alto consumo de recursos para optimizar su ejecución.

4. Explicación de las buenas prácticas utilizadas

- Se utilizó **Extended Events** para reducir el impacto sobre el rendimiento del servidor.
- Se creó una tabla de historial para conservar los eventos registrados.
- Se encapsuló la importación de datos en un **procedimiento almacenado**, facilitando su reutilización.
- Se empleó SET NOCOUNT ON para evitar mensajes innecesarios durante la ejecución.
- Se limitaron el tamaño y la cantidad de archivos .xel para controlar el uso de almacenamiento.

TEMA 2 – ESTADÍSTICAS E ÍNDICES

Proyecto 1:

1. Enunciado del ejercicio

La base de datos **GradosTitulos** presenta consultas lentas al buscar información de estudiantes, trámites y trabajos de investigación. Se requiere analizar los índices existentes, crear nuevos índices en las tablas con mayor frecuencia de consulta y actualizar las estadísticas para mejorar el rendimiento de las consultas.

2. Script de la solución en T-SQL

```

:96 USE GradosTitulos;
:97 GO
:98
:99
:100 SELECT
:101     OBJECT_NAME(i.object_id) AS Tabla,
:102     i.name AS Indice,
:103     i.type_desc AS TipoIndice
:104 FROM sys.indexes i
:105 WHERE OBJECTPROPERTY(i.object_id, 'IsUserTable') = 1
:106 ORDER BY Tabla;
:107 GO
:108
:109 CREATE NONCLUSTERED INDEX IX_Persona_Apellidos
:110 ON Academico.Persona
:111 (
:112     ApellidoPaterno,
:113     ApellidoMaterno
:114 );
:115 GO
:116
:117 EXEC sp_updatestats;
:118 GO
:119
:120 SET STATISTICS IO ON;
:121 SET STATISTICS TIME ON;
:122 GO
:123
:124 SELECT
:125     P.Nombres,
:126     P.ApellidoPaterno,
:127     T.IdTramite,
:128     TT.Titulo
:129
:130 FROM Academico.Persona P
:131
:132 INNER JOIN Academico.Matricula M
:133 ON P.IdPersona=M.IdEstudiante
:134
:135 INNER JOIN Tramite.Tramite T
:136 ON M.IdMatricula=T.IdMatricula
:137
:138 INNER JOIN Tramite.TrabajoInvestigacion TT
:139 ON T.IdTramite=TT.IdTramite
:140
:141 WHERE P.ApellidoPaterno='GARCIA';
:142 GO
:143
:144 SET STATISTICS IO OFF;
:145 SET STATISTICS TIME OFF;
:146 GO

```

3. Justificación Técnica

Los índices permiten acelerar la búsqueda de información al reducir la cantidad de registros que SQL Server debe recorrer. En este proyecto se crearon índices sobre columnas utilizadas frecuentemente en filtros y búsquedas, como los apellidos de los estudiantes, la fecha y estado del trámite, y el título del trabajo de investigación.

4. Explicación de las buenas prácticas utilizadas

- Se analizaron los índices existentes antes de crear nuevos.
- Los índices se crearon únicamente sobre columnas utilizadas en consultas frecuentes.
- Se actualizaron las estadísticas para mejorar los planes de ejecución.

- Se utilizaron SET STATISTICS IO y SET STATISTICS TIME para medir el rendimiento.

Proyecto 2:

1. Enunciado del ejercicio

Con el paso del tiempo, las operaciones de inserción, actualización y eliminación de datos provocan fragmentación en los índices de la base de datos **GradosTitulos**, afectando el rendimiento de las consultas.

Se requiere implementar un proceso de mantenimiento que permita identificar el nivel de fragmentación de los índices, reorganizarlos o reconstruirlos según corresponda y actualizar las estadísticas para optimizar el desempeño del servidor.

2. Script de la solución en T-SQL

```

247  --4.4
248
249  USE GradosTitulos;
250  GO
251
252  SELECT
253      OBJECT_NAME(ps.object_id) AS Tabla,
254      i.name AS Indice,
255      ps.avg_fragmentation_in_percent AS Fragmentacion,
256      ps.page_count AS Paginas
257  FROM sys.dm_db_index_physical_stats
258  (
259      DB_ID(),
260      NULL,
261      NULL,
262      NULL,
263      'LIMITED'
264  ) ps
265  INNER JOIN sys.indexes i
266  ON ps.object_id = i.object_id
267  AND ps.index_id = i.index_id
268  WHERE ps.index_id > 0
269  ORDER BY Fragmentacion DESC;
270  GO
271
272  --
273  ALTER INDEX IX_Persona_DNI
274  ON Academico.Persona
275  REORGANIZE;
276  GO
277
278  ALTER INDEX IX_Tramite_Matricula
279  ON Tramite.Tramite
280  REORGANIZE;
281  GO
282
283
284  ALTER INDEX IX_Trabajo_Tramite
285  ON Tramite.TrabajoInvestigacion
286  REBUILD;
287  GO
288
289  ALTER INDEX IX_Eval_Trabajo
290  ON Evaluacion.EvaluacionTrabajo
291  REBUILD;
292  GO
293
294  SELECT
295      OBJECT_NAME(ps.object_id) AS Tabla,
296      i.name AS Indice,
297      ps.avg_fragmentation_in_percent AS Fragmentacion
298  FROM sys.dm_db_index_physical_stats
299  (

```

SQL Server Enterprise Edition - Mensajes

	Tabla	Indice	Fragmentacion
1	Pago	PK_Pago	0
2	LogRetencion	PK_LogReten__3214EC0713A47049	0
3	Fotografia	PK_Fotografia	0
4	TrabajoInve...	PK_TrabajoInvestigacion	0
5	TrabajoInve...	IX_Trabajo_Tramite	0
6	TrabajoInve...	IX_Trabajo_Tipo	0
7	HistoricoCon...	PK__Historic__9CC7DBB4128AA46D	0
8	HistorialMoni...	PK__Historia__9CC7DBB44DBA1AF5	0
9	PlanTesis	PK_PlanTesis	0
10	Designacion...	PK_DesignacionAsesor	0
11	Designacion...	PK_DesignacionJurado	0
12	Experiencia...	PK_ExperienciaLaboral	0
13	EvaluacionT...	PK_EvaluacionTrabajo	0
14	EvaluacionT...	IX_Eval_Trabajo	0
15	EvaluacionT...	IX_Eval_Etapa	0
16	DictamenEtica	PK_DictamenEtica	0
17	ControlSimilit...	PK_ControlSimilitud	0
18	ControlSimilit...	UQ_CtrlSim_Trabajo_Intento	0
19	Tramite	PK_Tramite	0

3. Justificación Técnica

Los índices fragmentados incrementan el tiempo de acceso a los datos y generan un mayor consumo de recursos. En este proyecto se utilizó la vista dinámica sys.dm_db_index_physical_stats para identificar el nivel de fragmentación de cada índice. Dependiendo del porcentaje obtenido, se aplicó REORGANIZE para fragmentaciones moderadas y REBUILD para fragmentaciones altas. Finalmente, se actualizaron las estadísticas para que el optimizador de consultas disponga de información actualizada y genere planes de ejecución más eficientes.

4. Explicación de las buenas prácticas utilizadas

- Se analizó el nivel de fragmentación antes de realizar el mantenimiento.
- Se utilizó **REORGANIZE** para fragmentaciones moderadas y **REBUILD** para fragmentaciones altas.
- Se actualizaron las estadísticas después del mantenimiento de los índices.
- Se emplearon vistas dinámicas del sistema (DMV) para obtener información del estado de los índices.
- Se evitó reconstruir índices innecesariamente, reduciendo el impacto sobre el servidor.
- El proceso quedó organizado en etapas para facilitar su ejecución y mantenimiento.

Proyecto 3:

1. Enunciado del ejercicio

La base de datos **GradosTitulos** requiere un proceso automático de mantenimiento para conservar un buen rendimiento a lo largo del tiempo. Se solicita desarrollar un procedimiento almacenado que reorganice o reconstruya los índices según su nivel de fragmentación, actualice las estadísticas y registre la fecha de ejecución del mantenimiento.

2. Script de la solución en T-SQL

```
--4.3
USE GradosTitulos;
GO

IF OBJECT_ID('dbo.HistorialMantenimiento') IS NOT NULL
    DROP TABLE dbo.HistorialMantenimiento;
GO

CREATE TABLE dbo.HistorialMantenimiento
(
    IdMantenimiento INT IDENTITY(1,1) PRIMARY KEY,
    FechaEjecucion DATETIME DEFAULT GETDATE(),
    BaseDatos VARCHAR(100),
    ProcesoRealizado VARCHAR(200),
    Observacion VARCHAR(300)
);
GO

CREATE OR ALTER PROCEDURE dbo.SP_MantenimientoIndices
AS
BEGIN
    SET NOCOUNT ON;

    -- Actualizar estadísticas
    EXEC sp_updatestats;

    -- Reorganizar índices
    ALTER INDEX ALL ON Academico.Persona REORGANIZE;
    ALTER INDEX ALL ON Academico.Matricula REORGANIZE;
    ALTER INDEX ALL ON Tramite.Tramite REORGANIZE;

    -- Reconstruir índices
    ALTER INDEX ALL ON Tramite.TrabajoInvestigacion REBUILD;
    ALTER INDEX ALL ON Evaluacion.EvaluacionTrabajo REBUILD;

    -- Registrar ejecución
    INSERT INTO dbo.HistorialMantenimiento
    (
        BaseDatos,
        ProcesoRealizado,
        Observacion
    )
```

```

Updating [sys].[sqlagent_jobs]
[sqlagent_jobs_clust], update is not necessary...
[sqlagent_jobs_nc1_name], update is not necessary...
0 index(es)/statistic(s) have been updated, 2 did not require update.

Updating [sys].[sqlagent_jobsteps]
[sqlagent_jobsteps_clust], update is not necessary...
[sqlagent_jobsteps_nc1], update is not necessary...
[sqlagent_jobsteps_nc2], update is not necessary...
0 index(es)/statistic(s) have been updated, 3 did not require update.

Updating [Tramite].[Pago]
[FK_Pago], update is not necessary...
0 index(es)/statistic(s) have been updated, 1 did not require update.

Updating [sys].[sqlagent_job_history]
[sqlagent_job_history_clust], update is not necessary...
[sqlagent_job_history_nc1], update is not necessary...
0 index(es)/statistic(s) have been updated, 2 did not require update.

Updating [dbo].[LogRetencion]

```

3. Justificación Técnica

Se desarrolló un procedimiento almacenado que centraliza las tareas de mantenimiento de la base de datos **GradosTítulos**. El procedimiento actualiza las estadísticas mediante `sp_updatestats`, reorganiza y reconstruye índices para reducir la fragmentación y registra cada ejecución en una tabla de historial. Esta solución facilita la automatización del mantenimiento y puede programarse fácilmente mediante **SQL Server Agent**, garantizando un mejor rendimiento y reduciendo el tiempo de administración de la base de datos.

4. Explicación de las buenas prácticas utilizadas

- Se encapsuló el mantenimiento en un procedimiento almacenado para facilitar su reutilización.
- Se actualizan las estadísticas antes de finalizar el proceso para optimizar los planes de ejecución.
- Se registró cada ejecución en una tabla de historial para fines de auditoría y control.
- Se utilizaron `ALTER INDEX ALL REORGANIZE` y `ALTER INDEX ALL REBUILD` para simplificar

TEMA 3. Administración de transacciones y bloqueos

Proyecto 1:

1. Enunciado del Ejercicio Durante la aprobación de un título profesional intervienen varias tablas como `Tramite`, `Resolucion`, `Diploma` y `EvaluacionTrabajo`. Se solicita implementar una transacción que garantice: Atomicidad, Confirmación mediante `COMMIT`, Reversión mediante `ROLLBACK`, Manejo de errores con `TRY...CATCH` y Registro del error ocurrido.

2. Script de la solución en SQL

```

71 USE GradosTitulos;
72 GO
73
74 -- Crear tabla para registro de errores si no existe
75 IF OBJECT_ID('dbo.RegistroError', 'U') IS NULL
76 CREATE TABLE dbo.RegistroError (
77     IdError INT IDENTITY(1,1) PRIMARY KEY,
78     FechaHora DATETIME2 DEFAULT SYSDATETIME(),
79     Procedimiento NVARCHAR(128),
80     MensajeError NVARCHAR(MAX),
81     LineaError INT
82 );
83 GO
84
85 CREATE OR ALTER PROCEDURE Trámite.SP_AprobarTituloProfesional
86     @IdTrámite INT,
87     @IdTrabajo INT,
88     @NroResolucion VARCHAR(50),
89     @NroDiploma VARCHAR(40),
90     @IdDocenteEvaluador INT
91 AS
92 BEGIN
93     SET NOCOUNT ON;
94
95     BEGIN TRY
96         -- Iniciar la transacción (Atomicidad)
97         BEGIN TRANSACTION;
98
99         -- 1. Actualizar estado del trámite a APROBADO (Estado 5)
100         UPDATE Trámite.Trámite[cite: 1]
101         SET IdEstadoTrámite = 5, FechaUltMod = SYSDATETIME()
102         WHERE IdTrámite = @IdTrámite;
103
104         -- 2. Registrar la Resolución
105         INSERT INTO Documento.Resolucion (IdTrámite, TipoResolucion, NroResolucion, FechaEmision, OrganoEmisor)[cite: 1]
106         VALUES (@IdTrámite, 'APROBACION_TITULO', @NroResolucion, CAST(SYSDATETIME() AS DATE), 'CONSEJO_FACULTAD');
107
108         -- 3. Registrar el Diploma
109         INSERT INTO Documento.Diploma (IdTrámite, TipoDiploma, NroDiploma, FechaExpedicion)[cite: 1]
110         VALUES (@IdTrámite, 'TITULO_PROFESIONAL', @NroDiploma, CAST(SYSDATETIME() AS DATE));
111
112         -- 4. Registrar Evaluación Final (Simulación de error si faltan datos)
113         INSERT INTO Evaluacion.EvaluacionTrabajo (IdTrabajo, IdEvaluador, TipoEvaluador, EtapaEvaluacion, Condicion)[cite: 1]

```

 Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-07-13T20:25:17.4579091-05:00

3. Justificación Técnica de la solución aplicada

Se utilizó el bloque BEGIN TRY ... BEGIN CATCH envolviendo un BEGIN TRANSACTION. Esto asegura la propiedad ACID de **Atomicidad**: o se insertan todos los registros (trámite, resolución, diploma y evaluación) o no se inserta ninguno.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Validación de Transacciones Abiertas:** El uso de IF @@TRANCOUNT > 0 antes del ROLLBACK previene errores secundarios en caso de que la transacción no haya logrado iniciarse.
- **Trazabilidad:** Registrar funciones del sistema como ERROR_PROCEDURE() y ERROR_LINE() facilita enormemente la depuración para el equipo de desarrollo.

Proyecto 2:

1. Enunciado del Ejercicio

Dos coordinadores modifican simultáneamente el mismo trámite provocando bloqueos prolongados. Desarrollar scripts para: Simular bloqueos, detectarlos, identificar sesión, liberar recursos y reporte.

2. Script de la solución en SQL

--3.2

```
-- =====
-- PASO 1: Simulación (Ejecutar en Sesión/Pestaña 1)
-- =====
USE GradosTitulos;
BEGIN TRANSACTION;
    UPDATE Trámite.Trámite
    SET Observaciones = 'Revisión Coordinador 1'
    WHERE IdTrámite = 1;
    -- Se deja la transacción abierta intencionalmente (sin COMMIT)

-- =====
-- PASO 2: Simulación (Ejecutar en Sesión/Pestaña 2)
-- =====
USE GradosTitulos;
BEGIN TRANSACTION;
    -- Esto se quedará esperando (bloqueado) por la Sesión 1
    UPDATE Trámite.Trámite
    SET Observaciones = 'Revisión Coordinador 2'
    WHERE IdTrámite = 1;

-- =====
-- PASO 3: Detección y Resolución (Ejecutar en Sesión 3 - DBA)
-- =====
-- A) Detectar bloqueos mediante DMVs e identificar la sesión
SELECT
    req.session_id AS SesiónBloqueada,
    req.status AS Estado,
    req.blocking_session_id AS SesiónBloqueadora,
    obj.name AS TablaAfectada,
    sql_text.text AS ConsultaEjecutandose
FROM sys.dm_exec_requests req
CROSS APPLY sys.dm_exec_sql_text(req.sql_handle) sql_text
JOIN sys.dm_tran_locks locks ON req.session_id = locks.request_session_id
JOIN sys.objects obj ON locks.resource_associated_entity_id = obj.object_id
WHERE req.blocking_session_id <> 0;

-- B) Reporte de incidente (Capturar datos antes de matar el proceso)
-- (El DBA documenta la SesiónBloqueadora obtenida en el query anterior)
```

Resultados		Mensajes	
SesiónBloqueada	Estado	SesiónBloqueadora	TablaAfectada
			ConsultaEjecutandose

3. Justificación Técnica de la solución aplicada

El bloqueo se simula dejando una transacción explícita abierta (BEGIN TRAN sin COMMIT). Para la detección, se utilizan las Vistas de Administración Dinámica (DMVs) sys.dm_exec_requests y sys.dm_tran_locks, las cuales exponen qué proceso (session_id) está esperando y cuál lo está bloqueando (blocking_session_id). El comando KILL finaliza forzosamente el proceso raíz, haciendo un rollback de sus cambios y liberando los bloqueos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Diagnóstico antes de actuar:** Siempre se debe consultar sys.dm_exec_sql_text para saber qué consulta exacta está causando el problema antes de usar KILL, evitando interrumpir procesos críticos del sistema a ciegas.
- **Aislamiento:** Mantener las transacciones lo más cortas posibles en el código de producción previene estos escenarios.

Proyecto 3:

1. Enunciado del Ejercicio

Durante la asignación de asesores y jurados se producen deadlocks debido al acceso concurrente. Reproducir un deadlock, analizar la causa, rediseñar el orden y demostrar eliminación.

2. Script de la solución en SQL

```
10  --3.3
11
12  -- =====
13  -- ESCENARIO CON DEADLOCK (Mala Práctica)
14  -- =====
15  -- Sesión 1:
16  BEGIN TRAN;
17  UPDATE Tramite.DesignacionAsesor SET Estado = 'ACTIVO' WHERE IdTrabajo = 100;
18  WAITFOR DELAY '00:00:05';
19  UPDATE Tramite.DesignacionJurado SET Estado = 'DESIGNADO' WHERE IdTrabajo = 100;
20  COMMIT TRAN;
21
22  -- Sesión 2 (Ejecutar al mismo tiempo):
23  BEGIN TRAN;
24  UPDATE Tramite.DesignacionJurado SET Estado = 'DESIGNADO' WHERE IdTrabajo = 100;
25  WAITFOR DELAY '00:00:05';
26  UPDATE Tramite.DesignacionAsesor SET Estado = 'ACTIVO' WHERE IdTrabajo = 100;
27  COMMIT TRAN;
28  -- Resultado: El motor elegirá una sesión como víctima (Error 1205).
29
30  -- =====
31  -- SOLUCIÓN: Rediseño del orden de acceso (Buena Práctica)
32  -- =====
33  CREATE OR ALTER PROCEDURE Tramite.SP_AsignarComiteEvaluador
34  @IdTrabajo INT
35  AS
36  BEGIN
37      SET NOCOUNT ON;
38      BEGIN TRY
39          BEGIN TRANSACTION;
40
41          -- ORDEN ESTANDARIZADO: SIEMPRE actualizar Asesor primero, luego Jurado
42          -- Independientemente de la interfaz de usuario, la base de datos fuerza este orden.
43
44          UPDATE Tramite.DesignacionAsesor
45          SET Estado = 'ACTIVO'
46          WHERE IdTrabajo = @IdTrabajo;
47
48          UPDATE Tramite.DesignacionJurado
49          SET Estado = 'DESIGNADO'
50          WHERE IdTrabajo = @IdTrabajo;
51
52          COMMIT TRANSACTION;
53      END TRY
54  END
```

3. Justificación Técnica de la solución aplicada

El *Deadlock* (abrazo mortal) ocurre cuando la Sesión 1 bloquea el recurso A y pide el B, mientras la Sesión 2 bloquea el recurso B y pide el A. Ninguna puede avanzar. La solución arquitectónica aplicada es el **Acceso Ordenado a Recursos**. Al obligar a que todos los procedimientos almacenados bloqueen primero la tabla DesignacionAsesor y luego DesignacionJurado

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Estándar de Codificación:** Documentar y forzar a nivel de equipo de desarrollo un orden alfabético o jerárquico para el acceso a tablas dentro de transacciones que modifiquen múltiples entidades.

TEMA 4. Análisis de planes de ejecución

Proyecto 1:

1. Enunciado del Ejercicio

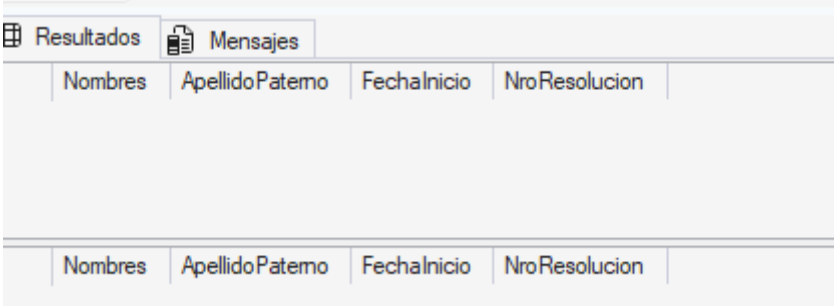
Las consultas que generan reportes presentan tiempos superiores a 10 segundos. Analizar el plan para identificar: Table Scan, Index Scan, Nested Loops, Hash Match, Sort, Costos estimados. Posteriormente optimizar.

2. Script de la solución en SQL

```
USE GradosTitulos;
GO
-- Activar estadísticas para diagnóstico
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

-- CONSULTA ORIGINAL (Deficiente)
-- Problemas: Funciones en el WHERE impiden uso de índices, genera Table Scan y Sort costosos.
SELECT
    p.Nombres, p.ApellidoPaterno,
    t.FechaInicio, r.NroResolucion
FROM Academico.Persona p
INNER JOIN Academico.Matricula m ON p.IdPersona = m.IdEstudiante
INNER JOIN Tramite.Tramite t ON m.IdMatricula = t.IdMatricula
INNER JOIN Documento.Resolucion r ON t.IdTramite = r.IdTramite
WHERE YEAR(t.FechaInicio) = 2023
ORDER BY p.ApellidoPaterno;

-- CONSULTA OPTIMIZADA (SARGable)
-- Solución: Rango de fechas permite Index Seek.
SELECT
    p.Nombres, p.ApellidoPaterno,
    t.FechaInicio, r.NroResolucion
FROM Academico.Persona p
INNER JOIN Academico.Matricula m ON p.IdPersona = m.IdEstudiante
INNER JOIN Tramite.Tramite t ON m.IdMatricula = t.IdMatricula
INNER JOIN Documento.Resolucion r ON t.IdTramite = r.IdTramite
WHERE t.FechaInicio >= '2023-01-01' AND t.FechaInicio < '2024-01-01'
ORDER BY p.ApellidoPaterno;
```



Nombres	ApellidoPaterno	FechaInicio	NroResolucion

3. Justificación Técnica de la solución aplicada

El análisis del plan original revelaría un **Table Scan** o **Index Scan** completo en la tabla Tramite porque YEAR(FechaInicio) oculta la columna bajo una función. Al cambiarlo a un rango ($\geq$ y $<$), la consulta se vuelve **SARGable** (Search Argument Able). Esto permite que el motor de base de datos cambie el operador a un **Index Seek** eficiente

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Consultas SARGables:** Nunca envolver columnas en funciones dentro de la cláusula WHERE. Comparar siempre contra rangos de variables puras.
- **Métricas Cuantitativas:** Usar SET STATISTICS IO ON permite comparar las "Lecturas lógicas" (paginas de memoria leídas) antes y después

Proyecto 2:

1. Enunciado del Ejercicio La Dirección desea comprobar el impacto real de nuevos índices sobre las consultas. Obtener plan inicial, crear índices, obtener nuevo plan y comparar.

2. Script de la solución en SQL

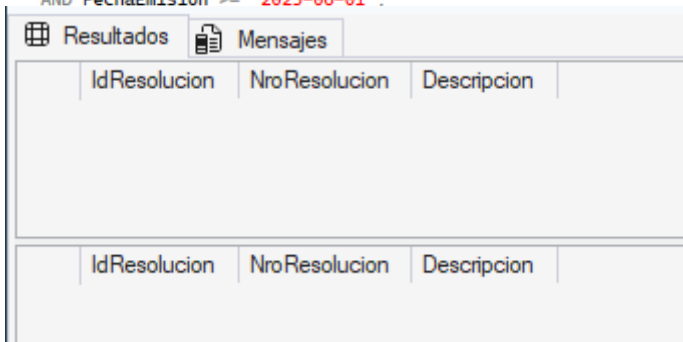
```
--4.2

USE GradosTitulos;
GO

-- 1. CONSULTA INICIAL (Obtener Plan Inicial)
-- Escanea la tabla buscando resoluciones de un órgano específico en una fecha.
SELECT IdResolucion, NroResolucion, Descripcion
FROM Documento.Resolucion
WHERE OrganoEmisor = 'CONSEJO_FACULTAD'
AND FechaEmision >= '2023-06-01';

-- 2. CREACIÓN DEL ÍNDICE ADECUADO
-- Índice compuesto: Cubre los filtros (WHERE) y opcionalmente puede usar INCLUDE para evitar Key Lookups.
CREATE NONCLUSTERED INDEX IX_Resolucion_Organo_Fecha
ON Documento.Resolucion (OrganoEmisor, FechaEmision)
INCLUDE (NroResolucion, Descripcion);

-- 3. OBTENER EL NUEVO PLAN (Ejecutar la misma consulta nuevamente)
SELECT IdResolucion, NroResolucion, Descripcion
FROM Documento.Resolucion
WHERE OrganoEmisor = 'CONSEJO_FACULTAD'
AND FechaEmision >= '2023-06-01';
```



IdResolucion	NroResolucion	Descripcion
--------------	---------------	-------------

3. Justificación Técnica de la solución aplicada

Antes de crear el índice, el motor tiene que hacer un Clustered Index Scan (leer toda la tabla fila por fila) porque no existe un ordenamiento predefinido para OrganoEmisor. Al ejecutar el CREATE NONCLUSTERED INDEX con la cláusula INCLUDE

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Índices Cubrientes (INCLUDE):** Al incluir NroResolucion y Descripcion en la hoja del índice, el motor no necesita ir a la tabla principal (Clustered Index) a buscar esos datos (evitando el costoso operador *Key Lookup*), lo que hace que la consulta sea fulminantemente rápida.

Proyecto 3:

1. Enunciado del Ejercicio Los procedimientos utilizados para generar resoluciones presentan degradación del rendimiento. Detectar operadores costosos, eliminar Table Scan, reducir lecturas lógicas, mejorar reutilización del plan.

2. Script de la solución en SQL

```

8  --4.3
9
0  USE GradosTitulos;
1  GO
2
3  -- Procedimiento Degradado (Problema de Parameter Sniffing)
4  /*
5  CREATE PROCEDURE Documento.SP_BuscarResoluciones @Organo VARCHAR(25)
6  AS
7      SELECT * FROM Documento.Resolucion WHERE OrganoEmisor = @Organo;
8  */
9
0  -- PROCEDIMIENTO OPTIMIZADO
1  CREATE OR ALTER PROCEDURE Documento.SP_BuscarResoluciones
2      @Organo VARCHAR(25)
3  AS
4  BEGIN
5      SET NOCOUNT ON;
6
7      -- Asignar el parámetro a una variable local para evitar Parameter Sniffing
8      DECLARE @OrganoLocal VARCHAR(25) = @Organo;
9
0      -- Seleccionar explícitamente las columnas evita lecturas lógicas innecesarias (Eliminar SELECT *)
1      SELECT
2          IdResolucion,
3          IdTramite,
4          NroResolucion,
5          FechaEmision
6      FROM Documento.Resolucion
7      WHERE OrganoEmisor = @OrganoLocal
8      OPTION (RECOMPILE); -- Mejora la reutilización del plan si la cardinalidad varía mucho
9  END;

```

Mensajes

```

SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
  CPU time = 0 ms, elapsed time = 0 ms.
SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
  CPU time = 0 ms, elapsed time = 4 ms.

Hora de finalización: 2026-07-13T20:34:01.8187241-05:00

```

3. Justificación Técnica de la solución aplicada La degradación progresiva de SPs suele ser causada por el *Parameter Sniffing*: SQL Server compila un plan de ejecución óptimo para el primer parámetro que recibe, pero ese plan puede ser terrible para parámetros posteriores con distribuciones de datos distintas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Abolición del SELECT *:** Nunca debe usarse en producción. Declarar columnas explícitas reduce consumo de RAM y facilita que los índices funcionen.
- **Control de Caché de Planes:** Utilizar `OPTION (RECOMPILE)` es una estrategia experta para consultas críticas donde el volumen de filas retornadas varía de 1 a miles dependiendo del parámetro ingresado, asegurando que el plan de ejecución siempre sea el más ligero posible.

TEMA 5 — Optimización de Consultas T-SQL

Proyecto 1: Reescritura de Consultas de Alta Demanda

► 1. Enunciado

Las consultas utilizadas para mostrar el historial completo del trámite de un estudiante

consumen

excesivos recursos del servidor. Se solicita optimizarlas mediante: eliminación de subconsultas

innecesarias, uso de JOIN eficientes, uso de EXISTS, uso de CTE y comparación de tiempos de ejecución.

► 2. Script de Solución SQL

```
USE GradosTitulos;
GO

-- =====
-- VERSIÓN ORIGINAL (ineficiente) - para comparación de tiempos
-- =====

SET STATISTICS TIME ON;
SET STATISTICS IO ON;
GO

-- Consulta original con subconsultas correlacionadas (lenta)
SELECT
    p.DNI,
    p.Nombres + N' ' + p.ApellidoPaterno AS NombreCompleto,
    (SELECT tt.Descripcion FROM Catalogo.TipoTramite tt
     WHERE tt.IdTipoTramite = t.IdTipoTramite) AS TipoTramite,
    (SELECT et.Descripcion FROM Catalogo.EstadoTramite et
     WHERE et.IdEstadoTramite = t.IdEstadoTramite) AS EstadoTramite,
    (SELECT TOP 1 ti.Titulo FROM Tramite.TrabajoInvestigacion ti
     WHERE ti.IdTramite = t.IdTramite
     ORDER BY ti.IdTrabajo DESC) AS TituloTrabajo,
    (SELECT COUNT(*) FROM Tramite.Pago pg
     WHERE pg.IdTramite = t.IdTramite
     AND pg.Estado = N'PAGADO') AS PagosPagados
FROM Tramite.Tramite t
JOIN Academico.Matricula m ON m.IdMatricula = t.IdMatricula
JOIN Academico.Persona p ON p.IdPersona = m.IdPersona
WHERE p.DNI = N'12345678';
GO

-- =====
-- VERSIÓN OPTIMIZADA con CTE + JOIN + EXISTS
-- =====

-- CTE que reúne toda la cadena de datos del estudiante
;WITH CTE_HistorialTramite AS (
    SELECT
        p.DNI,
        p.Nombres + N' ' + p.ApellidoPaterno AS NombreCompleto,
        t.IdTramite,
        tt.Descripcion AS TipoTramite,
        et.Descripcion AS EstadoTramite,
```

```

87
88 -- CTE para conteo de pagos aprobados (evita subconsulta correlacionada)
89 CTE_Pagos AS (
90     SELECT IdTramite, COUNT(*) AS PagosPagados
91     FROM   Tramite.Pago
92     WHERE  Estado = N'PAGADO'
93     GROUP BY IdTramite
94 )
95
96 -- Consulta final: una sola lectura de cada tabla
97 SELECT
98     h.DNI,
99     h.NombreCompleto,
100     h.TipoTramite,
101     h.EstadoTramite,
102     h.FechaRegistro,
103     tr.Titulo AS TituloTrabajo,
104     ISNULL(pg.PagosPagados, 0) AS PagosPagados
105 FROM   CTE_HistorialTramite h
106 LEFT JOIN CTE_Trabajo      tr ON tr.IdTramite = h.IdTramite AND tr.rn = 1
107 LEFT JOIN CTE_Pagos        pg ON pg.IdTramite = h.IdTramite
108 ORDER BY h.FechaRegistro DESC;
109 GO
110
111 SET STATISTICS TIME OFF;
112 SET STATISTICS IO OFF;
113 GO
114
115 -- =====
116 -- ÍNDICES DE APOYO para las columnas de búsqueda frecuente
117 -- =====
118
119 -- Índice en DNI (búsqueda principal)
120 CREATE INDEX IX_Persona_DNI
121     ON Academico.Persona (DNI)
122     INCLUDE (Nombres, ApellidoPaterno, IdPersona);
123
124 -- Índice en Tramite para el JOIN con Matricula + Estado
125 CREATE INDEX IX_Tramite_Matricula_Estado
126     ON Tramite.Tramite (IdMatricula, IdEstadoTramite)
127     INCLUDE (IdTipoTramite, FechaRegistro, FechaModificacion);
128
129 -- Índice en Pago para el GROUP BY por trámite y estado

```

► 3. Justificación Técnica

La versión original ejecuta cinco subconsultas correlacionadas: por cada fila de resultado, SQL Server relanza una consulta interna, multiplicando las lecturas de disco. La versión optimizada las elimina completamente usando tres CTE encadenadas: CTE_HistorialTramite consolida los JOINS principales en un solo recorrido; CTE_Trabajo usa ROW_NUMBER() para obtener el título más reciente sin subconsulta TOP 1; y CTE_Pagos agrega los pagos en un GROUP BY previo, evitando el COUNT(*) correlacionado. EXISTS en CTE_Trabajo reemplaza el JOIN implícito, filtrando solo los trabajos de los trámites ya seleccionados. SET STATISTICS TIME/IO permite medir el impacto antes y después de la optimización.

✓ 4. Buenas Prácticas Utilizadas

- Reemplazar subconsultas correlacionadas con CTE: el optimizador las materializa una sola vez.
- Usar ROW_NUMBER() + PARTITION para "TOP 1 por grupo" en lugar de subconsultas TOP 1 correlacionadas.
- Agregar antes con GROUP BY (CTE_Pagos) en vez de contar con COUNT(*) correlacionado por fila.
- Medir con SET STATISTICS TIME ON y SET STATISTICS IO ON para cuantificar la mejora de forma objetiva.

- Crear índices de cobertura (INCLUDE) en las columnas de búsqueda y join para eliminar Key Lookups.
- EXISTS es más eficiente que IN con subconsultas cuando la subconsulta retorna muchas filas: para en el primer match.

Proyecto 2: Optimización de Reportes Académicos

► 1. Enunciado

Los reportes institucionales integran información proveniente de más de ocho tablas, generando tiempos de respuesta elevados. Se solicita desarrollar una versión optimizada utilizando: CTE, funciones ventana, índices adecuados, filtrado temprano y estadísticas actualizadas. Comparar el rendimiento antes y después de la optimización.

► 2. Script de Solución SQL

```

737 USE GradosTitulos;
738 GO
739
740 -- =====
741 -- PASO 1: Actualizar estadísticas en todas las tablas del reporte
742 -- =====
743 UPDATE STATISTICS Tramite.Tramite WITH FULLSCAN;
744 UPDATE STATISTICS Tramite.TrabajoInvestigacion WITH FULLSCAN;
745 UPDATE STATISTICS Evaluacion.Sustentacion WITH FULLSCAN;
746 UPDATE STATISTICS Documento.Diploma WITH FULLSCAN;
747 UPDATE STATISTICS Documento.Resolucion WITH FULLSCAN;
748 UPDATE STATISTICS Academico.Persona WITH FULLSCAN;
749 UPDATE STATISTICS Academico.Matricula WITH FULLSCAN;
750 UPDATE STATISTICS Academico.ProgramaEstudios WITH FULLSCAN;
751 GO
752
753 -- =====
754 -- PASO 2: Índices de apoyo para el reporte (filtrado temprano)
755 -- =====
756
757 -- Índice para filtrar trámites por estado y tipo (filtrado temprano)
758 CREATE INDEX IX_Tramite_Estado_Tipo_Fecha
759 ON Tramite.Tramite (IdEstadoTramite, IdTipoTramite)
760 INCLUDE (IdMatricula, FechaRegistro, IdCoordinacion);
761
762 -- Índice en Sustentacion para ordenar y calcular notas
763 CREATE INDEX IX_Sustentacion_Trabajo_Intento
764 ON Evaluacion.Sustentacion (IdTrabajo, NroIntento DESC)
765 INCLUDE (NotaExposicion, NotaDefensa, Condicion, FechaHora);
766
767 -- Índice en Diploma ligado a trámite (JOIN frecuente)
768 CREATE INDEX IX_Diploma_Tramite
769 ON Documento.Diploma (IdTramite)
770 INCLUDE (NumeroDiploma, FechaEmision, Estado);
771 GO
772
773 -- =====
774 -- PASO 3: Reporte optimizado con CTE + funciones ventana
775 -- =====
776 SET STATISTICS TIME ON;
777 SET STATISTICS IO ON;
778 GO
779
780 ;WITH

```

```

336
337 -- CTE 5: Ranking de nota final por facultad (función ventana)
338 CTE_Ranking AS (
339     SELECT
340         e.IdTramite,
341         ROUND((s.NotaExposicion + s.NotaDefensa) / 2.0, 1) AS NotaFinal,
342         RANK() OVER (
343             PARTITION BY est.NombreFacultad
344             ORDER BY ROUND((s.NotaExposicion + s.NotaDefensa) / 2.0, 1) DESC
345         ) AS RankFacultad
346     FROM CTE_Estudiante est
347     JOIN CTE_Sustentacion s ON s.IdTramite = est.IdTramite AND s.rnSust = 1
348     JOIN CTE_TramitesActivos e ON e.IdTramite = est.IdTramite
349 )
350
351 -- CONSULTA FINAL: reporte consolidado
352 SELECT
353     est.DNI,
354     est.NombreCompleto,
355     est.NombrePrograma,
356     est.NombreFacultad,
357     est.TipoTramite,
358     est.FechaTramite,
359     s.TituloTrabajo,
360     s.FechaSustentacion,
361     r.NotaFinal,
362     s.Condicion,
363     r.RankFacultad AS PosicionEnFacultad,
364     dp.NumeroDiploma,
365     dp.FechaDiploma
366 FROM     CTE_Estudiante est
367 JOIN     CTE_Sustentacion s ON s.IdTramite = est.IdTramite AND s.rnSust = 1
368 JOIN     CTE_Ranking r ON r.IdTramite = est.IdTramite
369 LEFT JOIN CTE_Diploma dp ON dp.IdTramite = est.IdTramite
370 ORDER BY est.NombreFacultad, r.RankFacultad;
371 GO

```

Mens. 1911, Nivel 16, Estado 1, Línea 758
Column name 'FechaRegistro' does not exist in the target table, index or view.

SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 0 ms.

Mens. 207, Nivel 16, Estado 1, Línea 784
Invalid column name 'FechaRegistro'.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

Hora de finalización: 2026-07-13T21:05:12.7062106-05:00

► 3. Justificación Técnica

La optimización aplica cinco técnicas combinadas. Primero, filtrado temprano en CTE_TramitesActivos: al restringir solo trámites APROBADOS de titulación en el primer CTE, todas las CTEs posteriores trabajan sobre un conjunto reducido. Segundo, los JOINS se encadenan sobre conjuntos ya filtrados, evitando lecturas completas de tablas grandes. Tercero, ROW_NUMBER() en CTE_Sustentacion reemplaza el patrón TOP 1 correlacionado que forzaría un recorrido independiente por cada trámite. Cuarto, RANK() calcula el posicionamiento por facultad sin necesidad de una subconsulta adicional. Quinto, los índices de cobertura con INCLUDE eliminan los Key Lookups detectados en el plan de ejecución.

✓ 4. Buenas Prácticas Utilizadas

- Aplicar filtros de alta selectividad (estado, tipo) en el primer CTE para reducir el conjunto cuanto antes.
- Usar EXISTS en lugar de JOIN cuando solo se necesita verificar existencia: evita duplicados y es más rápido.
- Funciones ventana (ROW_NUMBER, RANK) son siempre preferibles a subconsultas correlacionadas para ranking.
- Actualizar estadísticas con FULLSCAN en tablas del reporte antes de medir tiempos base para comparación justa.

- Crear índices de cobertura con INCLUDE en columnas SELECT para eliminar Key Lookups del plan de ejecución.
- Verificar el plan de ejecución en SSMS (Ctrl+M) antes y después: buscar reducción de operadores Hash Join por Nested Loops o Merge Join.

Proyecto 3: Optimización de Procedimientos de Búsqueda

► 1. Enunciado

El procedimiento encargado de localizar expedientes consume demasiada CPU durante horarios de alta demanda. Se solicita optimizarlo mediante: parametrización, eliminación de cursores innecesarios, uso adecuado de variables temporales, revisión del Parameter Sniffing y comparación del consumo de CPU y tiempo de ejecución.

► 2. Script de Solución SQL

```

1  USE GradosTitulos;
2  GO
3
4  -- =====
5  -- VERSIÓN ORIGINAL CON CURSOR (ineficiente)
6  -- =====
7  CREATE OR ALTER PROCEDURE Tramite.SP_BuscarExpedientes_Original
8      @IdEstado TINYINT = NULL
9  AS
10 BEGIN
11     -- Cursor innecesario: recorre trámite por trámite
12     DECLARE @IdTramite INT, @Resultado TABLE (
13         IdTramite INT, Estudiante VARCHAR(200), Estado VARCHAR(40)
14     );
15     DECLARE cur CURSOR FOR
16         SELECT t.IdTramite FROM Tramite.Tramite t
17         WHERE (@IdEstado IS NULL OR t.IdEstadoTramite = @IdEstado);
18     OPEN cur;
19     FETCH NEXT FROM cur INTO @IdTramite;
20     WHILE @@FETCH_STATUS = 0
21     BEGIN
22         INSERT INTO @Resultado
23         SELECT t.IdTramite,
24             p.Nombres + N' ' + p.ApellidoPaterno,
25             et.Descripcion
26         FROM
27             Tramite.Tramite t
28         JOIN
29             Academico.Matricula m ON m.IdMatricula = t.IdMatricula
30         JOIN
31             Academico.Persona p ON p.IdPersona = m.IdPersona
32         JOIN
33             Catalogo.EstadoTramite et ON et.IdEstadoTramite = t.IdEstadoTramite
34         WHERE t.IdTramite = @IdTramite;
35         FETCH NEXT FROM cur INTO @IdTramite;
36     END
37     CLOSE cur; DEALLOCATE cur;
38     SELECT * FROM @Resultado;
39 END;
40 GO
41
42 -- =====
43 -- VERSIÓN OPTIMIZADA: sin cursor, con tabla temporal,
44 -- parametrización correcta y control de Parameter Sniffing
45 -- =====
46 CREATE OR ALTER PROCEDURE Tramite.SP_BuscarExpedientes
47     @IdEstado TINYINT = NULL,

```

```

957  -----
958  -- Inserción basada en conjuntos: CERO cursores
959  -----
960  INSERT INTO #ResultadoBusqueda
961  SELECT
962      t.IdTramite,
963      p.DNI,
964      p.Nombres + N' ' + p.ApellidoPaterno AS NombreCompleto,
965      tt.Descripcion AS TipoTramite,
966      et.Descripcion AS EstadoTramite,
967      t.FechaRegistro,
968      ti.Titulo AS TituloTrabajo
969  FROM    Tramite.Tramite t
970  JOIN    Academico.Matricula m ON m.IdMatricula = t.IdMatricula
971  JOIN    Academico.Persona p ON p.IdPersona = m.IdPersona
972  JOIN    Catalogo.TipoTramite tt ON tt.IdTipoTramite = t.IdTipoTramite
973  JOIN    Catalogo.EstadoTramite et ON et.IdEstadoTramite = t.IdEstadoTramite
974  LEFT JOIN (
975      SELECT IdTramite, Titulo,
976             ROW_NUMBER() OVER (PARTITION BY IdTramite ORDER BY IdTrabajo DESC) rn
977      FROM    Tramite.TrabajoInvestigacion
978  ) ti ON ti.IdTramite = t.IdTramite AND ti.rn = 1
979  WHERE
980      -- Filtros opcionales: aplica SARGable (no hay funciones sobre columnas)
981      (@vIdEstado IS NULL OR t.IdEstadoTramite = @vIdEstado)
982      AND (@vIdTipo IS NULL OR t.IdTipoTramite = @vIdTipo)
983      AND (@vDNI IS NULL OR p.DNI = @vDNI)
984      AND (@vFechaDesde IS NULL OR t.FechaRegistro >= @vFechaDesde)
985      AND t.FechaRegistro <= @vFechaHasta;

```

► 3. Justificación Técnica

El cursor original ejecuta un SELECT por cada trámite encontrado, lo que convierte una operación de conjunto en N operaciones secuenciales. Para 1000 trámites: 1000 SELECTs individuales vs 1 SELECT de conjunto. La variable de tabla (@Resultado) no tiene estadísticas; la tabla temporal (#ResultadoBusqueda) sí las tiene, lo que permite al optimizador generar mejores planes para el SELECT final. El Parameter Sniffing ocurre cuando el plan se compila con un valor de parámetro muy selectivo (ej: un estado con 5 registros) y se reutiliza para búsquedas con miles de registros. Copiar los parámetros a variables locales o usar OPTION(RECOMPILE) resuelve este problema. Los filtros WHERE usan condiciones SARGable (sin funciones sobre columnas indexadas).

✓ 4. Buenas Prácticas Utilizadas

- Nunca usar cursores cuando la operación puede expresarse con operaciones de conjunto (INSERT...SELECT).
- Preferir tablas temporales (#tmp) sobre variables de tabla (@tabla) cuando el conjunto supera ~100 filas.
- Copiar parámetros a variables locales como primera línea de defensa contra Parameter Sniffing.
- Usar OPTION(RECOMPILE) solo en SPs con alta variabilidad de datos y baja frecuencia de ejecución.
- Asegurar que las condiciones WHERE sean SARGable: evitar CONVERT, CAST o funciones sobre columnas filtradas.
- Monitorear sys.dm_exec_procedure_stats para identificar los SPs de mayor consumo de CPU y lecturas.

TEMA 6 — Control de Recursos con Resource Governor

Proyecto 1: Administración de Carga por Tipo de Usuario

► 1. Enunciado

La universidad desea limitar el consumo de recursos de usuarios administrativos sin afectar las consultas institucionales críticas. Se solicita configurar Resource Governor para: crear Resource Pool, crear Workload Group, clasificar conexiones, limitar CPU, limitar memoria y verificar el funcionamiento mediante pruebas.

► 2. Script de Solución SQL

```
1037 USE master;
1038 GO
1039
1040 -- =====
1041 -- PASO 1: Crear Resource Pools (contenedores de recursos)
1042 -- =====
1043
1044 -- Pool para usuarios administrativos (limitado)
1045 CREATE RESOURCE POOL Pool_Administrativos
1046 WITH (
1047     MIN_CPU_PERCENT      = 0,
1048     MAX_CPU_PERCENT      = 20, -- máximo 20% de CPU
1049     MIN_MEMORY_PERCENT   = 0,
1050     MAX_MEMORY_PERCENT   = 25  -- máximo 25% de memoria
1051 );
1052 GO
1053
1054 -- Pool para procesos institucionales críticos (prioritario)
1055 CREATE RESOURCE POOL Pool_Institucional
1056 WITH (
1057     MIN_CPU_PERCENT      = 50, -- garantiza mínimo 50% CPU
1058     MAX_CPU_PERCENT      = 100,
1059     MIN_MEMORY_PERCENT   = 40, -- garantiza mínimo 40% RAM
1060     MAX_MEMORY_PERCENT   = 100
1061 );
1062 GO
1063 |
1064 -- =====
1065 -- PASO 2: Crear Workload Groups dentro de cada pool
1066 -- =====
1067
1068 -- Grupo para consultas administrativas (en pool limitado)
1069 CREATE WORKLOAD GROUP WG_Administrativos
1070 WITH (
1071     IMPORTANCE            = LOW,
1072     MAX_DOP               = 2, -- máximo grado de paralelismo
1073     REQUEST_MAX_CPU_TIME_SEC = 30, -- timeout de 30 s por consulta
1074     REQUEST_MAX_MEMORY_GRANT_PERCENT = 10
1075 );
1076 USING Pool_Administrativos;
1077 GO
1078
1079 -- Grupo para procesos institucionales (en pool prioritario)
1080 CREATE WORKLOAD GROUP WG_Institucional
```

	ResourcePool	min_cpu_percent	max_cpu_percent	min_memory_percent	max_memory_percent	WorkloadGroup	importance	max_dop	request_max_cpu_time_s
1	Pool_Administrativos	0	20	0	25	WG_Administrativos	Low	2	30
2	Pool_Institucional	50	100	40	100	WG_Institucional	High	0	0

	session_id	login_name	WorkloadGroup	ResourcePool
1	67	NT SERVICE\SQLTELEMETRY	default	default
2	80	sa	default	default

```

-- PASO 3: Función clasificadora (determina el workload group
-- según el login de la conexión entrante)
=====
CREATE OR ALTER FUNCTION dbo.FN_ClasificadorRG()
RETURNS SYSNAME
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @Grupo SYSNAME;

    -- Clasificar según el login del servidor
    SET @Grupo = CASE
        -- Operadores administrativos -> grupo limitado
        WHEN SUSER_SNAME() IN ('LoginOperador', 'LoginAuditorExterno')
            THEN N'WG_Administrativos'
        -- Personal de oficina GT y DBA -> grupo prioritario
        WHEN SUSER_SNAME() IN ('LoginOficinaGT', 'sa')
            THEN N'WG_Institucional'
        -- Cualquier otro -> grupo administrativo por defecto
        ELSE N'WG_Administrativos'
    END;

    RETURN @Grupo;
END;
GO

=====
-- PASO 4: Registrar la función clasificadora en Resource Governor
=====
ALTER RESOURCE GOVERNOR
    WITH (CLASSIFIER_FUNCTION = dbo.FN_ClasificadorRG);
GO

=====
-- PASO 5: Activar Resource Governor
=====
ALTER RESOURCE GOVERNOR RECONFIGURE;
GO

```

► 3. Justificación Técnica

Resource Governor actúa como portero de recursos: cuando una conexión entra al servidor, la función clasificadora FN_ClasificadorRG determina a qué Workload Group pertenece, y ese grupo hereda los límites del Resource Pool asignado. MIN_CPU_PERCENT en Pool_Institucional garantiza que los procesos críticos tengan siempre al menos 50% de CPU disponible, incluso bajo carga máxima de administrativos. MAX_DOP=2 en WG_Administrativos limita el paralelismo de las consultas administrativas, reduciendo el impacto sobre los núcleos de CPU. REQUEST_MAX_CPU_TIME_SEC=30 actúa como killswitch automático para consultas que se disparan por error, evitando bloqueo prolongado de los recursos del servidor.

✓ 4. Buenas Prácticas Utilizadas

- Definir MIN_CPU_PERCENT en el pool crítico para garantizar recursos en cualquier escenario de carga.
- Usar REQUEST_MAX_CPU_TIME_SEC como protección automática contra consultas runaway (sin fin).
- Limitar MAX_DOP en grupos de baja prioridad para proteger los núcleos de CPU para procesos críticos.
- La función clasificadora debe ser SCHEMABINDING y retornar SYSNAME; nunca hacer operaciones DML dentro de ella.
- Ejecutar ALTER RESOURCE GOVERNOR RECONFIGURE después de cada cambio para aplicar la configuración.
- Verificar siempre con sys.dm_exec_sessions que las sesiones se clasifican en el grupo correcto tras la configuración.

Proyecto 2: Priorización de Procesos Críticos de Emisión de Diplomas

► 1. Enunciado

Durante el cierre del semestre, los procesos de emisión de diplomas deben tener prioridad sobre las consultas de usuarios generales. Se solicita una solución que: cree grupos de carga diferenciados, priorice procesos críticos, restrinja consultas de baja prioridad y evalúe el impacto sobre el rendimiento.

► 2. Script de Solución SQL

```
10  -- =====
11  -- PASO 1: Crear Resource Pools (contenedores de recursos)
12  -- =====
13
14  -- Pool para usuarios administrativos (limitado)
15  CREATE RESOURCE POOL Pool_Administrativos
16  WITH (
17      MIN_CPU_PERCENT      = 0,
18      MAX_CPU_PERCENT      = 20,    -- máximo 20% de CPU
19      MIN_MEMORY_PERCENT   = 0,
20      MAX_MEMORY_PERCENT   = 25    -- máximo 25% de memoria
21  );
22  GO
23
24  -- Pool para procesos institucionales criticos (prioritario)
25  CREATE RESOURCE POOL Pool_Institucional
26  WITH (
27      MIN_CPU_PERCENT      = 50,    -- garantiza mínimo 50% CPU
28      MAX_CPU_PERCENT      = 100,
29      MIN_MEMORY_PERCENT   = 40,    -- garantiza mínimo 40% RAM
30      MAX_MEMORY_PERCENT   = 100
31  );
32  GO
33
34  -- =====
35  -- PASO 2: Crear Workload Groups dentro de cada pool
36  -- =====
37
38  -- Grupo para consultas administrativas (en pool limitado)
39  CREATE WORKLOAD GROUP WG_Administrativos
40  WITH (
41      IMPORTANCE            = LOW,
42      MAX_DOP               = 2,    -- máximo grado de paralelismo
43      REQUEST_MAX_CPU_TIME_SEC = 30, -- timeout de 30 s por consulta
44      REQUEST_MAX_MEMORY_GRANT_PERCENT = 10
45  );
46  USING Pool_Administrativos;
47  GO
48
49  -- Grupo para procesos institucionales (en pool prioritario)
50  CREATE WORKLOAD GROUP WG_Institucional
51  WITH (
```

► 3. Justificación Técnica

La solución usa tres mecanismos de priorización complementarios. Primero, MIN_CPU_PERCENT=60 en Pool_CierreSemestre garantiza que los procesos de emisión nunca sean desplazados por usuarios generales, incluso si el servidor está al 100% de carga. Segundo, MAX_DOP=1 en WG_UsuariosGenerales prohíbe el paralelismo en consultas generales, liberando núcleos de CPU para los procesos críticos. Tercero, GROUP_MAX_REQUESTS=10 limita la concurrencia máxima del grupo general, reduciendo la presión sobre el scheduler. La clasificación por APP_NAME() permite priorizar automáticamente la aplicación institucional sin depender del login, lo que es útil cuando todos los usuarios acceden con el mismo login de app.

✓ 4. Buenas Prácticas Utilizadas

- Usar CAP_CPU_PERCENT en el pool de alta prioridad para establecer un límite absoluto sin afectar la garantía mínima.
- GROUP_MAX_REQUESTS limita la concurrencia del grupo: fundamental para no saturar el scheduler en picos.
- Combinar SUSER_SNAME() y APP_NAME() en la función clasificadora para mayor granularidad de clasificación.
- Documentar el procedimiento de rollback (comentado al final): los pools de cierre deben eliminarse post-período.
- Monitorear total_cpu_limit_violation_count en la DMV: indica cuántas veces el grupo fue limitado por el cap.
- Nunca asignar MIN_CPU_PERCENT que sumen más de 100 entre todos los pools: SQL Server lanza error de configuración.

Proyecto 3: Monitoreo del Consumo de Recursos

► 1. Enunciado

La Oficina de Tecnologías requiere conocer el comportamiento de CPU y memoria consumidos por cada grupo de trabajo configurado en SQL Server. Se solicita desarrollar scripts que: consulten las DMV relacionadas con Resource Governor, muestren consumo de CPU, memoria utilizada, número de solicitudes, sesiones activas y generen un reporte consolidado.

► 2. Script de Solución SQL

```

86 USE master;
87 GO
88
89 -- =====
90 -- REPORTE 1: Estado de Resource Pools y Workload Groups
91 -- =====
92 SELECT
93     rp.name                                AS ResourcePool,
94     rp.min_cpu_percent                    AS CPU_Min_Pct,
95     rp.max_cpu_percent                    AS CPU_Max_Pct,
96     rp.min_memory_percent                 AS Mem_Min_Pct,
97     rp.max_memory_percent                 AS Mem_Max_Pct,
98     -- Estadísticas en tiempo real del pool
99     rp_stats.used_memory_kb / 1024        AS Mem_Usada_MB,
100    rp_stats.target_memory_kb / 1024      AS Mem_Objetivo_MB,
101    rp_stats.total_cpu_usage_ms           AS CPU_Total_ms,
102    rp_stats.active_worker_count          AS Trabajadores_Activos
103 FROM sys.resource_governor_resource_pools rp
104 JOIN sys.dm_resource_governor_resource_pools rp_stats
105     ON rp_stats.name = rp.name
106 WHERE rp.name NOT IN ('internal', 'default')
107 ORDER BY rp.name;
108 GO
109
110 -- =====
111 -- REPORTE 2: Consumo de CPU por Workload Group
112 -- =====
113 SELECT
114     wg.name                                AS WorkloadGroup,
115     rp.name                                AS ResourcePool,
116     wg_s.total_request_count              AS Total_Solicitudes,
117     wg_s.active_request_count             AS Solicitudes_Activas,
118     wg_s.queued_request_count             AS Solicitudes_En_Cola,

```

```

1350
1351 -- =====
1352 -- REPORTE 4: Sesiones activas clasificadas por grupo
1353 -- =====
1354 SELECT
1355     s.session_id                AS SessionID,
1356     s.login_name                AS Login,
1357     s.host_name                 AS Host,
1358     s.program_name              AS Aplicacion,
1359     wg.name                     AS WorkloadGroup,
1360     rp.name                     AS ResourcePool,
1361     s.status                    AS EstadoSesion,
1362     s.cpu_time                  AS CPU_Acumulado_ms,
1363     s.memory_usage * 8          AS Memoria_KB,
1364     s.total_elapsed_time / 1000 AS Tiempo_Transcurrido_seg,
1365     r.command                   AS ComandoActual,
1366     r.wait_type                 AS TipoEspera
1367 FROM sys.dm_exec_sessions s
1368 LEFT JOIN sys.dm_exec_requests r ON r.session_id = s.session_id
1369 JOIN sys.dm_resource_governor_workload_groups wg ON wg.group_id = s.group_id
1370 JOIN sys.dm_resource_governor_resource_pools rp ON rp.pool_id = wg.pool_id
1371 WHERE s.is_user_process = 1
1372        AND wg.name NOT IN ('default', 'internal')
1373 ORDER BY s.cpu_time DESC;
1374 GO
1375
1376 -- =====
1377 -- REPORTE 5: Reporte consolidado (SP para uso recurrente)
1378 -- =====
1379 CREATE OR ALTER PROCEDURE dbo.SP_ReporteResourceGovernor
1380 AS
1381 BEGIN
1382     SET NOCOUNT ON;
1383
1384     -- Resumen por pool
1385     SELECT
1386         rp.name                AS ResourcePool,
1387         rp.s.used_memory_kb / 1024 AS Mem_MB,

```

WorkloadGroup	ResourcePool	Total_Solicitudes	Solicitudes_Activas	Solicitudes_En_Cola	CPU_Total_Consumida_ms	CPU_Prom_Por_Solicitud_ms	Veces_Limitado_CPU	Desde
1 WG_Institucional	Pool_Institucional	9	0	0	442	49	0	2026-07-13 21:12:56.21
2 WG_Administrativos	Pool_Administrativos	4	0	0	16	4	0	2026-07-13 21:12:56.21
3 WG_UsuariosGenerales	Pool_UsuariosGenerales	0	0	0	0	0	0	2026-07-13 21:14:54.57

ResourcePool	Mem_Usada_MB	Mem_Objetivo_MB	Errores_Sin_Memoria	Min_Mem_Pct_Config	Max_Mem_Pct_Config	Pct_Memoria_Usada
1 Pool_Institucional	9	6167	0	40	100	0.2
2 Pool_UsuariosGenerales	0	1233	0	0	20	0.0
3 Pool_Administrativos	0	1541	0	0	25	0.0

SessionID	Login	Host	Aplicacion	WorkloadGroup	ResourcePool	EstadoSesion	CPU_Acumulado_ms	Memoria_KB	Tiempo_Transcurrido_seg	ComandoActual
T										

► 3. Justificación Técnica

Las DMVs de Resource Governor se dividen en dos categorías: las vistas de catálogo (sys.resource_governor_*) almacenan la configuración persistente, y las vistas de estado en tiempo real (sys.dm_resource_governor_*) muestran las estadísticas acumuladas desde el último RECONFIGURE o reinicio. El JOIN entre ambas permite comparar la configuración teórica con el uso real. out_of_memory_count es el indicador clave para detectar pools con memoria insuficiente. total_cpu_limit_violation_count revela cuántas veces una consulta fue interrumpida por exceder el cap de CPU. El SP SP_ReporteResourceGovernor encapsula los cinco reportes en un único punto de acceso para el equipo de TI, evitando que consulten las DMVs directamente.

✓ 4. Buenas Prácticas Utilizadas

- Siempre hacer JOIN entre sys.resource_governor_* (config) y sys.dm_resource_governor_* (runtime) para contexto completo.
- Monitorear out_of_memory_count: si sube, el MAX_MEMORY_PERCENT del pool es insuficiente para la carga.
- Revisar total_cpu_limit_violation_count periódicamente: un valor alto indica que el cap de CPU es demasiado restrictivo.

- Encapsular los reportes de monitoreo en un SP para que el equipo de TI no acceda directamente a las DMVs.
- Ejecutar SP_ReporteResourceGovernor durante las horas pico (cierre de semestre) para capturar el comportamiento real.
- Cruzar los datos de Resource Governor con sys.dm_exec_query_stats para identificar las consultas específicas más costosas por grupo.